
CALAIS EXECUTION ALGORITHM

CANDIDATE PROJECT REPORT

Haoran Wu

June 2026

ABSTRACT

This report describes a compact execution algorithm for Binance USD-M Futures BTCUSDT Perpetual. It accepts a final target position, a permitted active execution price range, a target duration, and either a Chase or TWAP execution mode. The design priority is small-but-correct execution behavior rather than broad venue coverage: one symbol, one exchange family, one-way mode, deterministic simulator coverage, and a credential-gated Binance Testnet path. The core safety argument is an exposure invariant that accounts for confirmed fills, live open quantity, pending submits, pending cancels, unknown create outcomes, and proposed child quantity before any new order can be submitted. Deterministic simulator runs cover races that Testnet cannot reliably force, including fill-after-cancel and ambiguous create-timeout reconciliation. Accepted sanitized Binance Testnet artifacts are included for both Chase and TWAP; they preserve exchange order IDs, client order IDs, order/trade events, reconciliation output, and terminal metrics while redacting private account balance and position fields.

1 Introduction

The project brief asks for a Python execution algorithm for Binance USD-M Futures BTCUSDT Perpetual. The algorithm receives a final account target position, a price interval, a target duration, and an execution algorithm. The most important interpretation is that `target_position` is not an order quantity. The system first reads the current position and computes:

$$required_trade_quantity = target_position - current_position. \quad (1)$$

A negative-to-positive target therefore requires trading through zero. For example, moving from -0.003 BTC to $+0.005$ BTC requires buying 0.008 BTC, not 0.005 BTC. This rule is intentionally placed at execution creation, before algorithm selection, so Chase and TWAP share the same target-position semantics.

The second important interpretation is that price, time, and quantity cannot all be guaranteed across every market path. If a buy price range ends at an upper bound, the implementation cannot legally force a fill above that bound. If the market never becomes executable within the interval, a correct result may be partial or fully unfilled. The report therefore evaluates the project by whether the system reports actual outcome and preserves safety, not whether every run reaches 100 percent completion.

The implementation follows a “small and correct” strategy. It avoids multi-asset routing, durable production recovery, and mainnet automation. In exchange, the core order lifecycle is explicit, deterministic tests cover abnormal event ordering, and every child submit passes through the same exposure gate. The central design objective is not guaranteed completion. Price, time, and quantity cannot all be guaranteed under every market path. The objective is instead to make the outcome **correct, bounded, auditable, and explainable**: never knowingly overfill, never double-count a fill, never retry an ambiguous create with a fresh client order ID before reconciliation, and never report completion when price or exchange rules make completion illegal.

The scoring rubric makes this emphasis natural. Execution correctness and edge cases dominate the score, while report writing is a smaller but important communication channel. The report is therefore structured as a correctness defense. It starts with requirements and ownership boundaries, then spends most of its space on state, safety invariants, failure

Table 1: Submission status on the final report branch.

Area	Status
Core implementation	FastAPI service, execution engine, Chase, TWAP, simulator, Binance USD-M adapter, structured artifacts, and verification scripts are implemented.
Deterministic safety evidence	Simulator artifacts cover normal Chase, TWAP schedule, cancel/fill race, create-timeout reconciliation, duplicate events, stale data, price bounds, dust, and cross-zero target handling.
Accepted Binance Testnet evidence	Sanitized Chase and TWAP bundles contain non-empty Binance exchange order IDs, client order IDs, private stream events, reconciliation outputs, and terminal summaries.
Rejected connectivity artifacts	Older insufficient-margin Testnet attempts are retained only as connectivity/error evidence and are not counted as accepted order-lifecycle proof.
Submission controls	Mainnet mutation is hard-disabled by default; Testnet order sends require credentials and an explicit confirmation flag.

Table 2: How the report maps to the visible scoring categories.

Scoring area	Report emphasis
Execution correctness	Target position, Decimal math, price bounds, exposure invariant, create-timeout handling.
Edge cases and state	Parent and child state machines, cancel/fill race, duplicate events, stale streams, deadline behavior.
Testing quality	Deterministic simulator, T1-T10 matrix, raw artifacts, repeatable verification commands.
Algorithm design	Chase repricing policy and TWAP absolute schedule with carry-forward deficit.
Architecture and communication	Clear ownership boundaries, limitations, AI disclosure, and sanitized Testnet evidence status.

cases, and evidence. The goal is to make the implementation easy to challenge in an interview: each important claim should point to a module boundary, invariant, test, artifact, or limitation.

2 Architecture and Ownership Boundaries

The architecture is organized around one rule: the engine owns trading correctness. API and runtime code supervise execution, algorithms compute narrow decisions, and exchange adapters normalize venue behavior. They do not independently mutate exposure.

Design rule. Algorithms propose a price, schedule quantity, or reprice decision; the engine decides whether the resulting child order is safe to submit. This rule keeps the most dangerous responsibilities—filled quantity, live exposure, pending cancel quantity, unknown create outcomes, and terminal state—inside one component instead of spreading them across runtime callbacks and algorithm helpers.

`ExecutionRuntime` is the API-facing supervisor. It builds the correct environment, rejects concurrent active executions for the same environment and symbol, advances nonterminal executions, supervises market and private streams for Binance-like adapters, renews listen keys, and reconciles after stream events or disconnects.

`ExecutionService` is deliberately thin. It delegates execution lifecycle to `ExecutionEngine`, which owns parent execution state, child order state, exposure buckets, fill accounting, cancel flow, create-timeout handling, deadline behavior, and terminal summaries. This boundary keeps correctness checks in one place.

Chase and TWAP helpers do not submit orders. Chase computes the passive desired price and whether repricing is due. TWAP computes schedule progress and deficit. Both route through the engine’s submit path, so price checks, exposure checks, Decimal normalization, and unknown-order reservations remain common.

The concurrency model is intentionally single-process. The runtime rejects a second active execution for the same environment and symbol, and the engine serializes mutations on each execution record so `run_once`, manual cancel, stream-event application, and reconciliation do not race each other inside the process. This does not claim production-grade distributed ownership; it gives the assignment a clear correctness boundary that can be tested deterministically.

`ExchangeAdapter` gives the simulator and Binance adapter the same narrow contract: symbol rules, position, best bid/ask, submit, cancel, exact order lookup, stream events, reconciliation, and stream health. This is important because deterministic simulator tests exercise the same engine logic used by the Binance Testnet runner. The simulator is therefore a programmable venue for forced races, not a separate happy-path mock.

Table 3: Compact requirement coverage. More detailed edge-case and test mappings are in the appendix.

Requirement area	Evidence or note
Target-position semantics	Engine computes net delta from current position; no-action and cross-zero cases are tested.
Chase and TWAP	Chase passive repricing and TWAP absolute schedule are implemented as narrow algorithm helpers.
Price bounds and Decimal math	Decimal-string API fields, Decimal internal math, side-aware rounding, tick/step/min checks.
State machines and overfill safety	Parent and child states are monotonic; submit path enforces the exposure invariant.
Deterministic simulator	Simulator covers partial fill, cancel/fill race, post-only rejection, create timeout, duplicate/lower fills, stale data, and stream disconnect.
Binance Testnet integration code	Adapter and runner implement symbol rules, REST mutation, exact lookup, user streams, and evidence artifacts.
Accepted Binance Testnet Chase/TWAP evidence	Accepted sanitized Chase and TWAP artifacts are included with non-empty exchange order IDs.
Final reporting and AI disclosure	This LaTeX report and AI_USAGE.md document decisions, evidence, and verification.

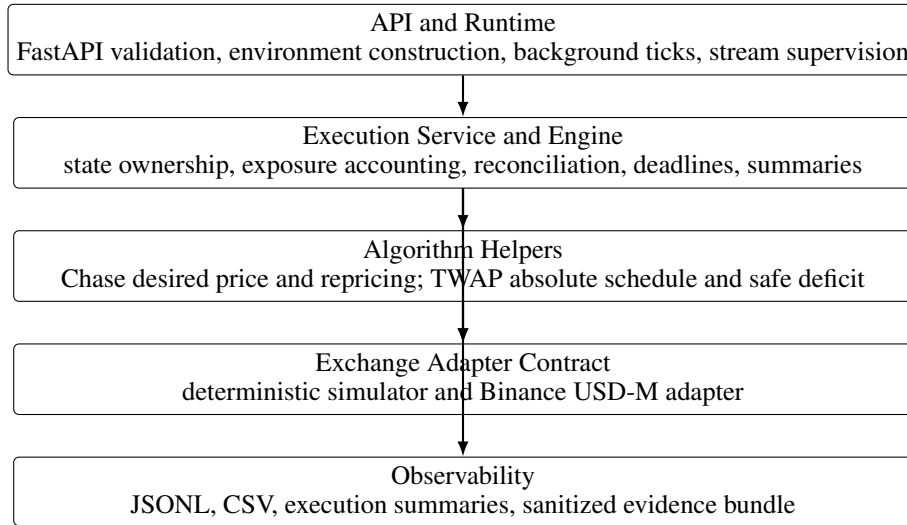


Figure 1: Layered ownership. The algorithm layer proposes; the engine decides whether the proposed child order is safe.

3 Lifecycle and State Machines

The parent execution lifecycle is monotonic. Terminal executions do not return to RUNNING. The engine serializes operations per execution so API requests, background ticks, stream events, manual cancel, and reconciliation cannot concurrently mutate the same execution record.

The child state machine is designed for disagreement between REST responses and user-stream events. A fill can arrive after a cancel request. A create request can time out while the exchange still accepts the order. A REST snapshot can be older than a private stream event. The engine therefore treats cumulative fills as monotonic, keeps pending-cancel and unknown-create exposure reserved, and uses exact client-order lookup to repair ambiguous create outcomes.

Auditability. Parent execution IDs, internal child order IDs, and Binance client order IDs are intentionally related. A parent execution such as `exec_3168600ee25b4193` produces client order IDs with a short deterministic prefix such as `ce_3168600ee25b_1`. This makes logs, child-order CSVs, fill CSVs, reconciliation rows, and Binance order evidence joinable without relying on timestamps alone. It also scopes reconciliation to the execution under review; broad prefix scans can help diagnostics, but exact client-order lookup is the proof used for ambiguous create outcomes.

4 Core Safety Invariant

The central correctness claim is the submit gate:

Phase	Allowed next phase	Purpose
CREATED	VALIDATING	Request accepted and domain objects built.
VALIDATING	RUNNING or terminal	Position, symbol rules, bounds, duration, and dust checks.
RUNNING	CANCELLING or terminal	Submit, cancel, reconcile, deadline, and fill processing.
CANCELLING	terminal	Stop active exposure, reconcile, then summarize.
Terminal	none	COMPLETED, PARTIALLY COMPLETED, EXPIRED, CANCELLED, or FAILED.

Figure 2: Parent execution state transition table. Terminal states are one-way.

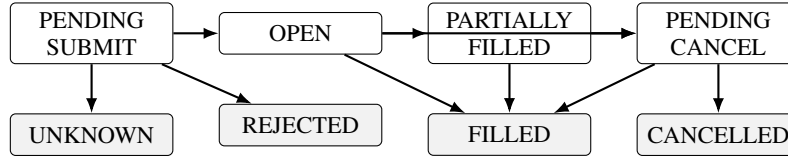


Figure 3: Child order states. UNKNOWN is a reserved-exposure state, not a harmless error.

$$\begin{aligned}
 & \text{confirmed_filled} + \text{live_open} + \text{pending_submit} \\
 & + \text{pending_cancel} + \text{unknown_order} + \text{new_child_quantity} \\
 & \leq \text{normalized_target_trade_quantity} + \text{permitted_tolerance}.
 \end{aligned} \tag{2}$$

This is stronger than recomputing a simple remaining quantity. `confirmed_filled` is parent-level cumulative filled quantity. `live_open` can still trade at the venue. `pending_submit` protects the interval between local intent and exchange response. `pending_cancel` protects the period where a cancel request has been sent but fills can still arrive. `unknown_order` protects ambiguous create outcomes. `new_child_quantity` is the proposed order.

The two most important buckets are `pending_cancel` and `unknown_order`. If the system releases `pending_cancel` too early, it can submit a replacement while the old order is still fillable. If it releases `unknown_order` after a create timeout, it can retry with a fresh client order ID even though the original order may be live. Both cases can create predictable overfill. Equation 2 prevents this by reserving exposure until reconciliation proves it is gone.

Other invariants support this gate:

- Parent confirmed filled quantity is monotonic.
- Per-child cumulative filled quantity is monotonic.
- Duplicate trade or event messages cannot increase parent fill twice.
- A timed-out create remains UNKNOWN until exact `origClientId` reconciliation.
- No aggressive child order may violate the configured price bound.
- Terminal execution state cannot return to RUNNING.

5 Chase Design

Chase uses passive limit orders at the current best queue price: buy orders at best bid and sell orders at best ask. Passive orders are post-only when the venue supports it. Repricing requires both a price movement threshold and a minimum interval:

$$\text{reprice_difference_bps} = \left| \frac{\text{desired_price}}{\text{active_order_price}} - 1 \right| \times 10000. \tag{3}$$

The default repricing mode is `ADVERSE_ONLY`. A buy order reprices upward when the best bid moves up enough; a sell order reprices downward when the best ask moves down enough. This default is deliberately conservative. It avoids unnecessary cancels when the existing order is already passive at a more favorable price and reduces cancel storm risk. `TWO_SIDED` is still available for experiments or interview discussion, but the safer default is to avoid churn unless the market moves away from the resting order.

Table 4: Exposure buckets used by the submit gate.

Bucket	Meaning	When it is released or moved
confirmed_filled	Parent cumulative fill already executed at the venue.	Never decreases; contributes to completion and reduces remaining target quantity.
live_open	Accepted order quantity still executable at the venue.	Moves to confirmed fill after fill events, or is released after cancel/reconciliation proves it is no longer live.
pending_submit	Local submit intent before a definitive venue response.	Moves to live open, rejected, or unknown after the submit response or timeout classification.
pending_cancel	Quantity on an order with a cancel request in flight.	Stays reserved until cancel, fill, or reconciliation gives a terminal child state.
unknown_order	Quantity from an ambiguous create outcome.	Released only after exact <code>origClientId</code> lookup proves absence, or moved back to live/filled if found.
new_child_quantity	The proposed additional child order.	Accepted only if Equation 2 remains within the normalized target quantity.

Partial fills are handled at the parent level. If an order partially fills and then a reprice is needed, replacement quantity is not the old child’s original quantity and not merely the venue-reported remaining quantity. It is the target quantity minus parent confirmed fills minus all still-reserved exposure. This is the same submit path as Equation 2, so cancel/fill race behavior is handled by the engine rather than by Chase-specific logic.

Deadline behavior is explicit. Under `CANCEL_REMAINDER`, active orders are cancelled and actual unfilled quantity is reported. Under `AGGRESSIVE_WITHIN_RANGE`, the engine may attempt a bounded marketable limit: buy no higher than the upper bound and sell no lower than the lower bound. If the market is outside range, the correct result is partial or unfilled, not fake completion.

6 TWAP Design

TWAP is implemented as an absolute schedule, not a sleep loop. At monotonic elapsed time t :

$$scheduled_cumulative_quantity(t) = total_trade_quantity \times \frac{elapsed_time}{total_duration}. \quad (4)$$

$$quantity_deficit(t) = scheduled_cumulative_quantity(t) - confirmed_cumulative_filled_quantity(t). \quad (5)$$

Before submitting, the engine subtracts reserved exposure:

$$safe_child_quantity = quantity_deficit - (live_open + pending_submit + pending_cancel + unknown_order). \quad (6)$$

This design solves the problems that make the invalid `slice_qty / sleep` implementation unsafe. Absolute schedule time avoids accumulated sleep drift. Unfilled earlier quantity naturally carries forward into the next deficit. Open or unknown earlier slices cannot be submitted twice because reserved exposure is subtracted. The final slice can absorb legal step-size rounding remainder, but the invariant still prevents exceeding the normalized target.

The scheduling window ends at `target_duration_seconds`. After that point the engine does not create new passive scheduled TWAP children. Deadline handling is a separate bounded cleanup phase. Under `CANCEL_REMAINDER`, cleanup cancels and reconciles remaining exposure. Under `AGGRESSIVE_WITHIN_RANGE`, cleanup may first cancel passive exposure and then submit a marketable limit child, but only within the configured price bound and only if the exposure invariant permits it. The summary therefore reports `decision_deadline_elapsed_seconds`, `terminal_cleanup_duration_seconds`, `total_lifecycle_duration_seconds`, and post-deadline order counts.

For example, suppose a 0.010 BTC buy is scheduled over 100 seconds. At 40 seconds the planned cumulative quantity is 0.004 BTC. If confirmed fills are only 0.001 BTC and an earlier child still reserves 0.001 BTC, the schedule deficit is

0.003 BTC but the safe new child quantity is 0.002 BTC. The missing earlier quantity is carried forward, while the still-open quantity is not submitted twice. This is the important difference between a schedule deficit and a child order size.

The deterministic TWAP artifact at `reports/evidence/simulation/twap/exec_61fadac604f4440a` shows the same schedule behavior. Over a 100 second target, it submitted five 20-second slices of 0.002 BTC each, reconciled every slice fill, and completed exactly 0.010 BTC without exceeding the scheduled deficit.

The accepted Testnet TWAP run at `reports/evidence/testnet/twap/exec_85bef3985ea3431a` also records the planned cumulative quantity, submitted quantity, open quantity, filled quantity, and unfilled quantity for each slice. Its first two slices were below the legal order shape and therefore submitted nothing; later slices carried the deficit forward, submitted legal child orders, and completed the required 0.004 BTC with `overflow_quantity=0`.

7 Precision, Risk, and Binance Integration

All trading prices and quantities use `Decimal`. API trading fields are decimal strings, and Binance parameters are serialized as strings. This matters because direct Python `float` construction can introduce non-exchange-valid prices or quantities before tick and step rounding.

Risk validation is side-aware. Passive buys must not round through the best ask or exceed the upper bound. Passive sells must not round through the best bid or go below the lower bound. Aggressive buy deadline orders cannot exceed the upper bound, and aggressive sell deadline orders cannot go below the lower bound. Quantity normalization floors to step size rather than rounding up into over-trading; dust is recorded or rejected according to the implemented rule.

The Binance adapter reads dynamic BTCUSDT symbol rules from exchange metadata, including tick size, quantity step, minimum quantity, minimum notional, trading status, and supported time-in-force assumptions [Binance, 2026]. Market data is based on best bid/ask updates. Private user-data events and REST reconciliation repair order and fill state. REST is also used for initialization, create/cancel mutation, exact order lookup, and reconciliation after disconnects.

Create and cancel mutations are classified conservatively. HTTP 408 and transport timeout on create map to an unknown create outcome; the child remains UNKNOWN and its quantity remains reserved. HTTP 408 and transport timeout on cancel keep the child in pending-cancel exposure until reconciliation. Mainnet mutation is hard disabled by default and requires explicit configuration; it is not part of the take-home demonstration path.

Secrets are runtime-only. API keys, API secrets, request signatures, listen keys, and signed payloads are sanitized from artifacts and are not committed.

8 Testing and Evidence

The deterministic simulator is mandatory because Testnet cannot reliably produce all failure modes. It can script market data, partial fills, fill during cancel, cancel delay, post-only rejection, create timeout where the order actually exists, duplicate or lower cumulative fill snapshots, stale market data, stream disconnects, and reconciliation snapshots. These tests are repeatable and run without credentials.

Table 5: Required scenario coverage. File names identify the main evidence source; many scenarios also have unit-level regressions.

ID	Scenario	Evidence
T1	Normal Chase	<code>scripts/run_sim_chase.py</code> ; engine and simulator order tests.
T2	Chase Reprice	Chase threshold/min-interval tests; summary metric coverage.
T3	Partial Fill + Cancel Race	<code>scripts/run_sim_cancel_race.py</code> ; cancel-race simulator tests.
T4	Create Timeout	<code>scripts/run_sim_create_timeout.py</code> ; exact lookup and unknown exposure tests.
T5	TWAP Carry-forward	TWAP absolute schedule and deficit tests.
T6	Tail Quantity	Decimal, dust, min-quantity, and step-size tests.
T7	Price Outside Range	Price-bound and deadline/outside-range tests.
T8	Stream Disconnect	Runtime stream restart, reconcile, and stale-market tests.
T9	Duplicate Event	Duplicate trade and cumulative-fill monotonicity tests.
T10	Cross-zero Position	Required-trade and account-position tests.

Current reproducible non-live baseline verification passed after the TWAP deadline clarification:

uv run pytest -q tests/unit tests/simulation → 511 passed

Credentialed/network-enabled integration verification is separate: `uv run pytest -q tests/integration`. Those results should be reported only for a full credentialed local review where the Binance integration tests run successfully.

The four simulator scripts also ran successfully during this report build and wrote committed structured artifacts:

- `reports/evidence/simulation/chase/exec_c8dc942476764355`
- `reports/evidence/simulation/twap/exec_61fadac604f4440a`
- `reports/evidence/simulation/cancel-race/exec_669d47a536a94682`
- `reports/evidence/simulation/create-timeout/exec_3ddb8a47995348d0`

The repository contains Binance Testnet runner scripts for Chase and TWAP. They require credentials and an explicit `-confirm-send-orders` flag and never fall back to simulation. Table 6 summarizes the accepted sanitized Testnet evidence now included in the repository.

Table 6: Accepted Binance Testnet evidence.

Algorithm	Evidence bundle	What it proves
Chase	<code>testnet/chase/exec_3168600ee25b4193</code> ; client ID <code>ce_3168600ee25b_1</code> ; exchange order ID <code>16277695886</code> .	Accepted order lifecycle evidence: private user-stream events were observed, execution-matching order events were applied, final reconciliation found one order and one fill, and the execution completed with COMPLETED.
TWAP	<code>testnet/twap/exec_85bef3985ea3431a</code> ; client IDs <code>ce_85bef3985ea3_1</code> , <code>ce_85bef3985ea3_2</code> ; exchange order IDs <code>16277882286</code> , <code>16277899689</code> .	Accepted TWAP evidence: two exchange orders were observed, private user-stream events were applied, the slice ledger shows carried-forward schedule deficit, final reconciliation found two orders and one fill, and the execution completed with COMPLETED.

These accepted artifacts preserve request snapshots, symbol rules, execution logs, timelines, child-order rows, fills, reconciliation output, evidence manifests, and terminal summaries while redacting `ACCOUNT_UPDATE` balance and position fields. The older credentialed raw Testnet runs under `reports/evidence/testnet/chase` and `reports/evidence/testnet/twap` are retained only as rejected connectivity/error artifacts. Both reached Binance and failed before order acceptance with `BINANCE-2019:Margin is insufficient.`; both manifests have `accepted_exchange_order_evidence=false` and empty `exchange_order_ids`.

9 Results and Metrics

Each execution summary is intended to reconstruct quantity, price, order, time, and safety behavior. Table 7 shows selected deterministic run outputs from the committed evidence bundle under `reports/evidence/simulation`.

Table 7: Selected simulator results from current verification.

Run	Result
Normal Chase	<code>chase/exec_c8dc942476764355</code> , child <code>ce_c8dc94247676_1</code> : one passive buy child for 0.010 BTC at 50000.00, then a simulator fill and reconciliation to COMPLETED.
TWAP schedule	<code>twap/exec_61fadac604f4440a</code> : five 0.002 BTC slices over 100 seconds filled and reconciled to 0.010 BTC total with <code>overflow_quantity=0</code> .
Cancel/fill race	<code>cancel-race/exec_669d47a536a94682</code> : a 0.004 BTC fill arrived during cancel; replacement was 0.006 BTC, preserving total exposure at 0.010.
Create timeout	<code>create-timeout/exec_3ddb8a47995348d0</code> , child <code>ce_3ddb8a479953_1</code> : unknown exposure was 0.010 before exact reconciliation and 0 after the original order was found open; no fresh client order ID was used.

Table 8 gives the terminal metrics from the accepted Testnet summaries. These are not substitutes for the deterministic race tests; they prove that the same service path can read Binance rules and quotes, submit exchange-accepted orders, receive private events, reconcile by client/exchange order identifiers, and produce final execution metrics.

Table 8: Accepted Testnet terminal metrics.

Metric	Chase	TWAP
Execution ID	exec_3168600ee25b4193	exec_85bef3985ea3431a
Initial to target position	0 → 0.001 BTC	0.0010 → 0.005 BTC
Required / filled / unfilled	0.001/0.001/0 BTC	0.004/0.004/0 BTC
Orders / cancels / reprices	1/0/0	2/1/0
Arrival mid / VWAP	62213/62212.5	62092.35/62092.9
Slippage bps	-0.0804	0.0886
Overfill / price violations	0/0	0/0
Exchange order IDs	16277695886	16277882286, 16277899689
Requested / lifecycle duration	12s/4.672s	12s/17.295s
Decision / cleanup duration	4.672s/0s	12s/5.295s
Post-deadline scheduled / aggressive children	0/0	0/1
Final status / reason	COMPLETED; TARGET_QUANTITY_FILLED	COMPLETED; TARGET_QUANTITY_FILLED

The accepted TWAP Testnet run used a 12s requested scheduling window and a 17.295s total lifecycle. The difference is deadline cleanup: the first passive child was cancelled, private user-stream events were applied, and a bounded IOC child completed the remaining target. No passive scheduled TWAP child was submitted after the 12s scheduling deadline. One deadline-aggressive child was submitted during cleanup, inside the configured price range, and completed the target with overfill_quantity=0.

The cancel/fill race artifact is the clearest overfill evidence. The initial child was 0.010 BTC. During cancel, 0.004 BTC filled. The replacement child was only 0.006 BTC. The final summary showed:

Metric	Value
required quantity	0.010 BTC
confirmed filled	0.004 BTC
live open	0.006 BTC
reserved exposure	0.006 BTC
unknown order	0 BTC
client order IDs	ce_669d47a536a9_1, ce_669d47a536a9_2

The create-timeout artifact demonstrates idempotency. The first child entered UNKNOWN; a subsequent run before reconciliation did not generate a new client order ID. Exact lookup found the original order and moved exposure back to live open. The final reason was CREATE_TIMEOUT_RECONCILED.

These examples also explain why completion rate below 100 percent can be correct. If price bounds, deadline policy, minimum quantity, post-only rejection, or venue state prevent a legal completion, the system should return actual filled and unfilled quantity with a clear reason.

10 Real Failure Case and Hardening

The most important failure case found during development was ambiguous Binance create timeout handling. A timeout from POST /fapi/v1/order does not prove the order failed. The exchange may have accepted it while the client lost the response. If the client clears exposure and retries with a fresh client order ID, both the original and replacement can fill.

The fix was to classify create HTTP 408 and transport timeout as unknown mutation outcomes. The child stays in UNKNOWN; its quantity remains in unknown_order_quantity; the engine blocks new child creation that would violate the invariant; and reconciliation queries the exact origClientId. If the order is found, it is promoted back to live open or filled state. If exact lookup proves it does not exist, unknown exposure is cleared. Broad prefix scans are useful diagnostics, but they are not proof that a specific timed-out order does not exist.

Table 9: Create-timeout hardening.

Unsafe behavior	Implemented correction
Treat timeout as failure and submit a replacement with a fresh client order ID. Clear unknown exposure using only a broad prefix scan or stale order list. Assume user stream and REST responses arrive in causal order.	Keep the child in UNKNOWN, reserve its quantity, and block any replacement that would exceed the exposure invariant. Query the exact original client order ID and reconcile that child before releasing or moving exposure. Apply cumulative fills monotonically and allow REST reconciliation to repair stream gaps or out-of-order delivery.

External-review hardening also improved several related paths:

- Partial fills returned in create responses now update parent fills immediately.
- Lower or duplicate cumulative snapshots cannot reduce or double-count parent fills.
- Stale market data pauses execution with a controlled reason instead of raising unexpectedly.
- Runtime unknown-reconciliation failures are recorded and retried.
- User stream events and disconnects trigger bounded execution-scoped reconciliation.
- Shutdown stops new child creation, cancels active orders, reconciles, and leaves no background execution task running.

11 Limitations and Future Work

The implementation is intentionally compact. Table 10 separates accepted take-home scope from production work that should not be hidden inside this submission.

Table 10: Known limitations and next steps.

Limitation	Current consequence	Professional next step
In-memory execution state	Process restart loses execution records and cannot replay partially completed jobs.	Add durable event storage, startup replay, and restart-safe reconciliation before production use.
Single-process ownership	No distributed symbol/account lock or multi-worker leader election.	Add explicit execution ownership, lease renewal, and failover semantics.
BTCUSDT / one-way mode only	Hedge-mode position-side semantics are rejected rather than modeled.	Design a separate target-position invariant for hedge mode before enabling it.
Limited operational surface	Stream supervision and artifact logs exist, but there are no production dashboards, alerts, or runbooks.	Add health metrics, alerting, operator actions, and documented incident procedures.
Testnet dependency	Future live reruns depend on Binance Testnet funding, permissions, symbol state, and risk checks.	Keep deterministic simulator as the race oracle and maintain accepted Testnet bundles as venue-integration evidence.
Mainnet disabled by default	Mainnet configuration compatibility exists, but automated live trading is intentionally not demonstrated.	Require a separate production readiness review, controls, and rollout plan before mainnet mutation.

The older raw credentialed rejection artifacts under `reports/evidence/testnet/chase` and `reports/evidence/testnet/twap` are retained only as rejected connectivity/error artifacts. Simulator artifacts should not be substituted for accepted Testnet artifacts, and accepted Testnet artifacts should not be overstated as deterministic race coverage. The two evidence sources answer different questions: the simulator forces edge cases; Testnet proves real venue integration on accepted orders.

12 AI Usage

AI assistance was used for planning, code review, test design, documentation drafting, and report preparation. Suggestions were treated as drafts. The implementation was checked against the project brief, corrected where unsafe, and verified with automated tests plus deterministic simulator scripts. The main reviewed areas were create-timeout handling, unknown exposure, cancel/fill race safety, cumulative fill monotonicity, duplicate event handling, Decimal-only price and quantity paths, TWAP carry-forward, and documentation alignment.

The candidate remains responsible for explaining and modifying the submitted code. No secrets, credentials, request signatures, listen keys, or private Binance account data are included in the repository.

A Edge-Case Matrix

Table 11: Brief edge cases and implemented behavior.

ID	Edge case	Implemented behavior
E1	Target already reached	No child order; terminal no-action/completed behavior.
E2	Target below minimum order size	Reject or record dust explicitly; never silently round up into overfill.
E3	Partial fill then cancel/reprice	Replacement quantity is based on parent confirmed fills and reserved exposure.
E4	Fill during pending cancel	Pending-cancel exposure remains reserved until terminal state or reconciliation.
E5	Create request timeout	Child becomes UNKNOWN; exact client-order lookup before clearing exposure.
E6	Duplicate or out-of-order events	Trade IDs and monotonic cumulative quantity prevent double counting and state regression.
E7	Price jumps outside range	Stop active chasing; deadline policy reports actual unfilled quantity.
E8	Frequent price movement	Reprice threshold and minimum interval limit cancel storm risk.
E9	Stream disconnect or stale quote	Pause new action, reconnect, reconcile, then resume or exit safely.
E10	Deadline with remainder	Cancel remainder or make bounded aggressive attempt within range; report actual result.
E11	Post-only reject	Refresh state and retry within bounded behavior; no infinite loop.
E12	Target crosses zero	Required trade is net position delta, not close/open split confusion.

B Artifact Checklist

Final submission should include source code, tests, scripts, README, AI disclosure, this report, and selected evidence artifacts. Simulator artifacts under `reports/evidence/simulation` should include request snapshots, execution logs, execution summaries, child-order CSVs, fill CSVs, timelines, and TWAP ledgers. The accepted sanitized Testnet artifacts under `reports/evidence/testnet/chase/exec_3168600ee25b4193` and `reports/evidence/testnet/twap/exec_85bef3985ea3431a` additionally include `symbol_rules.json`, `reconciliation_orders.csv`, and `evidence_manifest.json` with non-empty exchange order IDs and accepted exchange-order evidence. The older insufficient-margin artifacts remain only rejected connectivity/error evidence.

Artifacts must not include API secrets, listen keys, request signatures, signed payloads, or private account data not needed for review. Accepted Testnet logs redact `ACCOUNT_UPDATE` balance and position fields while preserving order/trade updates needed to prove exchange acceptance.

References

Binance. Binance usd-m futures api documentation. <https://developers.binance.com/docs/derivatives/usds-margined-futures/>, 2026. Reference for venue concepts used by the implementation.